

VOYAGEWRIGHT

Continuous Development and Mainline Integration Standard

Project Architecture, Phase Boundaries, Parallel Development,
Mainline Convergence, and Incremental Delivery Governance

GOVERNING PRINCIPLE

Parallel projects are encouraged. Parallel phase stacks are prohibited by default. The unit of implementation is the phase. The unit of integration is the completed phase. The authoritative development baseline is current accepted **main**.

Version 1.0 | August 8, 2026
Permanent Voyagewright Engineering Standard

Document Control

Field	Value
Status	Governing baseline
Applies to	All Voyagewright projects, phases, developers, Codex agents, integrators, CI and release work
Primary authority	Phase architecture, parallel development, mainline safety, dependencies, migration coordination, convergence
Companion authority	Global Product Governance governs product reality. Sounding Line governs verification and release evidence.
Core rule	Every phase is independently mainline-safe and converges promptly after acceptance.

Authority Stack

Authority	Question
Global Product Governance	What must be true for the human-facing product?
This Standard	How must projects and phases evolve and converge?
Sounding Line	How is implementation proven and release-authorized?
Project Governing Document	What does this subsystem own and become?
Phase Contract	What exact independently mergeable increment is delivered now?

Contents

1. Executive Summary
2. Problem Statement and Historical Motivation
3. Governing Authority and Precedence
4. Canonical Development Topology
5. Non-Negotiable Principles
6. Vocabulary and State Model
7. Mainline Safety Contract
8. Accepted Phase Integration Patterns
9. Phase Boundary Design Rules
10. Parallel Development Model
11. Concurrency Classes and Decision Rules
12. Dependency Governance
13. Branch and Worktree Lifecycle
14. Mainline Advancement During Active Work
15. Reconciliation and Semantic Conflict Resolution
16. Schema and Migration Governance
17. Shared Contract and Canonical Ownership
18. Feature Activation and Dormancy
19. Compatibility, Cutover, and Retirement
20. Phase Completion States
21. Mainline Acceptance Procedure
22. Post-Merge Proof and Cleanup
23. Sounding Line Integration and Evidence Reuse
24. Integration Debt and Branch Aging
25. Failure, Recovery, and Restart Policy
26. Production, Staging, and Release Relationship
27. Machine-Readable Development Control Plane
28. Mandatory Future Phase Prompt Contract
29. Codex and Agent Governance
30. Human Integrator Governance
31. Cross-Project Parallelism Examples
32. Anti-Patterns and Prohibited Practices
33. Metrics and Engineering Health
34. Adoption and Transition Plan

35. Change Control and Exceptions

36. Final Acceptance Criteria

Appendices: Mainline Safety Contract; Active Phase Registration; Migration Reservation; Reconciliation Report; Integration Debt; Phase Acceptance Checklist

1. Executive Summary

This section governs **phase-complete integration; incomplete projects versus incomplete behavior; production separation**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

2. Problem Statement and Historical Motivation

This section governs **long-lived divergence; semantic merge conflicts; delayed test and schema drift**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

3. Governing Authority and Precedence

This section governs **product governance; Sounding Line; project documents; current origin/main**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

4. Canonical Development Topology

This section governs **short-lived phase branches; continuously advancing main; next phase from integrated main**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

5. Non-Negotiable Principles

This section governs **current-main truth; prompt convergence; semantic ownership; isolation; evidence**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

6. Vocabulary and State Model

This section governs **project; phase; increment; locally complete; mainline candidate; accepted mainline; integration debt**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

7. Mainline Safety Contract

This section governs **post-phase capability; active and dormant behavior; compatibility; rollback; permanent-stop test**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

8. Accepted Phase Integration Patterns

This section governs **additive foundation; compatible migration; vertical slice; dormant foundation; cutover; retirement**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

9. Phase Boundary Design Rules

This section governs **stable plateaus; bounded ownership; explicit non-goals; no future-phase dependency for current correctness**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

10. Parallel Development Model

This section governs **one active unmerged phase per project; parallel projects; disjoint internal lanes**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

11. Concurrency Classes and Decision Rules

This section governs **Class A independent; B coordinated; C serialized shared authority; D hard dependency**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

12. Dependency Governance

This section governs **hard versus soft dependencies; no branch-as-platform-truth; no circular phase dependency**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

13. Branch and Worktree Lifecycle

This section governs **fresh origin/main; dedicated worktree; checkpoint commits; historical branch after acceptance**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

14. Mainline Advancement During Active Work

This section governs **fetch; inspect intervening commits; classify overlap; reconcile before acceptance**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

15. Reconciliation and Semantic Conflict Resolution

This section governs **canonical ownership; clean Git merge is insufficient; reconciliation record**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

16. Schema and Migration Governance

This section governs **phase-scoped reservations; SQLite/MySQL parity; expand-migrate-contract; backfill; rollback**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

17. Shared Contract and Canonical Ownership

This section governs **single canonical owner; provider seams; identity, runtime, navigation, privacy, release authorities**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

18. Feature Activation and Dormancy

This section governs **main may contain incomplete projects; no incoherent exposed UI; governed feature flags**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

19. Compatibility, Cutover, and Retirement

This section governs **thin adapters; observation windows; additive cutover; retirement conditions**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

20. Phase Completion States

This section governs **IMPLEMENTED**; **FOCUSED_VALIDATED**; **LOCALLY_COMPLETE**; **READY_FOR_OWNER_WALKTHROUGH**; **MAINLINE_CANDIDATE**; **ACCEPTED_MAINLINE**; **RELEASED**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

21. Mainline Acceptance Procedure

This section governs **ancestry; reconciliation; Sounding Line; merge; post-merge proof; parity**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

22. Post-Merge Proof and Cleanup

This section governs **actual integrated SHA; leases; temp resources; worktree retirement; cleanup receipt**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

23. Sounding Line Integration and Evidence Reuse

This section governs **source-bound evidence; invalidation; targeted reruns; finalizer authority**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

24. Integration Debt and Branch Aging

This section governs **visible debt; owner; expiry; aging risk; no unexplained completed branches**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

25. Failure, Recovery, and Restart Policy

This section governs **no destructive Git; preserve evidence; restart from current main when obsolete**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

26. Production, Staging, and Release Relationship

This section governs **main versus staging versus production; immutable releases; rollback; database compatibility**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

27. Machine-Readable Development Control Plane

This section governs **project registry; active phases; migration reservations; dependency graph; integration debt**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

28. Mandatory Future Phase Prompt Contract

This section governs **preflight; scope; non-goals; safety contract; dependencies; migrations; validation; merge behavior**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

29. Codex and Agent Governance

This section governs **governed participant; preserve work; no phase stacking; truthful completion language**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

30. Human Integrator Governance

This section governs **semantic review; owner walkthrough; no convenience merges; escalation**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

31. Cross-Project Parallelism Examples

This section governs **Admiralty with Tideglass; Drydock with Shipwright; shared identity serialization; Landfall provider dependency**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

32. Anti-Patterns and Prohibited Practices

This section governs **mega-branches; giant convergence week; unreserved migrations; duplicate authorities; hidden broken surfaces**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

33. Metrics and Engineering Health

This section governs **branch age; local-complete-to-main time; integration debt; conflicts; migration collisions; regressions**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

34. Adoption and Transition Plan

This section governs **apply to new phases first; register active work; add machine-readable policy; Sounding Line enforcement**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

35. Change Control and Exceptions

This section governs **written exception; owner; scope; risk; expiry; compensating controls; governance amendment**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

36. Final Acceptance Criteria

This section governs **all future phases governed; no default phase stacks; main continuously coherent; large convergence exceptional**. The rule is normative for all future Voyagewright work unless a documented, time-bounded exception has been approved.

The implementation objective is to keep the integrated product continuously truthful. A phase may add substantial new capability, but it may not rely on an unmerged future phase to make the current mainline coherent. The design must therefore separate foundations, compatibility, activation, and retirement into boundaries that can be accepted independently.

Before coding, the responsible agent records ownership, dependencies, schema impact, shared contracts, activation state, rollback behavior, and the exact mainline-safe end condition. During implementation, checkpoint commits may preserve work, but they do not change the authority state of the phase. Before acceptance, current origin/main is fetched and reviewed again.

If main has advanced, the phase classifies the intervening changes and reconciles semantic overlap. Sounding Line then determines which evidence remains valid and which evidence must be rerun. The phase cannot claim mainline acceptance until the integrated SHA itself has passed the required gate and remote parity is proven.

The governing bias is toward small, reversible, additive change. Destructive changes, shared-authority rewrites, and broad migrations require stronger coordination because they reduce the amount of safe parallelism available to other projects.

Required declaration	Minimum content
Ownership	Canonical owner and all shared consumers
Dependencies	Hard, soft, or none; producing phase if applicable
Mainline safety	What remains valid if later phases never occur
Schema impact	Models, migrations, backfill, compatibility window
Activation	Active, dormant, feature-gated, compatibility-only, or retirement
Evidence	Sounding Line scope required for acceptance
Failure path	Rollback, disable, or restart strategy

Appendix A - Mainline Safety Contract Template

This template is normative. Projects may add fields, but may not omit the governing information represented here without an approved exception.

Field	Required content
Post-phase capability	What useful capability exists after this phase?
Active behavior	What becomes active immediately?
Dormant behavior	What remains intentionally unavailable?
Compatibility	What old behavior remains supported and for how long?
Future work	What later phases remain intentionally unfinished?
Schema impact	What models, migrations, and backfills change?
Rollback/disable	How can the phase be safely disabled or reverted?
Permanent-stop test	Would main remain coherent if no later phase were implemented?

Use Rule

The template must describe implementation truth, not the intended future state. Unknown information must be marked unknown and resolved before the associated acceptance gate.

Appendix B - Active Phase Registration Template

This template is normative. Projects may add fields, but may not omit the governing information represented here without an approved exception.

Field	Required content
Project / phase	Stable identity
Branch / worktree	Task-owned development location
Base SHA	Exact accepted mainline used to start
State	PLANNED through ACCEPTED_MAINLINE
Integration pattern	Foundation, migration, vertical slice, dormant, cutover, retirement
Concurrency class	A, B, C, or D
Dependencies	Producing phases and contracts
Shared contracts	Canonical owners touched
Schema reservation	SQLite/MySQL ranges or none

Use Rule

The template must describe implementation truth, not the intended future state. Unknown information must be marked unknown and resolved before the associated acceptance gate.

Appendix C - Migration Reservation Template

This template is normative. Projects may add fields, but may not omit the governing information represented here without an approved exception.

Field	Required content
Reservation owner	Project and phase
SQLite range	Reserved identifiers
MySQL range	Reserved identifiers
Models owned	Canonical models controlled by the phase
Shared models	Models requiring coordination
Backfill	Dry-run, commit, verify requirements
Compatibility	Old/new reader and writer window
Rollback	Application rollback and data restore implications

Use Rule

The template must describe implementation truth, not the intended future state. Unknown information must be marked unknown and resolved before the associated acceptance gate.

Appendix D - Reconciliation Report Template

This template is normative. Projects may add fields, but may not omit the governing information represented here without an approved exception.

Field	Required content
Original base	Phase starting SHA
Current main	Fetches mainline SHA
Intervening commits	Commits reviewed
Overlap class	Unrelated, adjacent, direct, authority-changing
Semantic decisions	How canonical ownership was preserved
Files/contracts adapted	Exact reconciliation scope
Evidence invalidated	Sounding Line determination
Residual debt	Any deferred compatibility or cleanup

Use Rule

The template must describe implementation truth, not the intended future state. Unknown information must be marked unknown and resolved before the associated acceptance gate.

Appendix E - Integration Debt Template

This template is normative. Projects may add fields, but may not omit the governing information represented here without an approved exception.

Field	Required content
Debt item	Stable ID and description
Completed phase	Work being withheld
Reason	Why prompt convergence is not currently safe
Owner	Responsible person/project
Risk	What can drift while waiting
Expiry	Date or mainline condition
Convergence plan	Exact trigger and required proof
Escalation	What happens when expiry is reached

Use Rule

The template must describe implementation truth, not the intended future state. Unknown information must be marked unknown and resolved before the associated acceptance gate.

Appendix F - Phase Acceptance Checklist

This template is normative. Projects may add fields, but may not omit the governing information represented here without an approved exception.

Field	Required content
Preflight	Fresh origin/main; clean worktree; registered phase
Implementation	Scope complete; non-goals preserved
Safety	Mainline Safety Contract satisfied
Reconciliation	Current main reviewed and conflicts resolved
Validation	Sounding Line required evidence valid
Integration	Merged to canonical mainline
Post-merge	Integrated SHA validated as required
Parity	Local/remote main 0 ahead / 0 behind
Cleanup	Leases, temp data, ports, and worktree handled
Next phase	May start only from accepted integrated main

Use Rule

The template must describe implementation truth, not the intended future state. Unknown information must be marked unknown and resolved before the associated acceptance gate.

Final Governing Rule

BUILD CONTINUOUSLY. INTEGRATE CONTINUOUSLY.

Parallel projects are encouraged when ownership and resources permit. Parallel phase stacks are prohibited by default. Every accepted phase must leave main coherent, and every next phase must begin from the actual integrated product.